Uzlezz Engine · АИД · ИТМО · ЛР 3 «Рендер» · вариант Д

Отсев и подача на пальцах

Как движок перестал рисовать всё подряд: объекты регистрируются в подсистеме отсева, каждый кадр она отвечает «что видно камере», а рендер сортирует видимое и подаёт его одним пакетом. Простыми словами, шаг за шагом, у каждого шага указаны файлы с кодом.

Если надо объяснить за 30 секунд

Раньше рендер каждый кадр проходил по всем объектам сцены и рисовал каждый. В сцене из 10 000 объектов камера видит примерно 1 800, а работа шла на все 10 000.

Теперь у движка есть **реестр видимости** (`Visibility::Service`). Объект попадает туда один раз и обновляется, только когда его сдвинули. Каждый кадр рендер спрашивает реестр: «что видит эта камера?», получает список видимых и собирает из них **пакет**. Пакет сортируется так, чтобы одинаковые текстуры шли подряд, а внутри от ближних к дальним, и только потом рисуется.

Итог на Windows: кадр 21,43 → 7,01 мс, в 3,05 раза быстрее. Смен текстур за кадр 10 000 → 16. Картинка при этом не изменилась: это проверено сравнением кадров.

Как читать этот файл

Простыми словами

Синий блок: суть на бытовом примере.

Где в коде

Серый блок: файл и функция, где это сделано. Открывай и смотри.

Если спросят

Жёлтый блок: короткие ответы на вероятные вопросы защиты.

Все пути от корня репозитория. Код написал однокурсник, коммит `54eb67e` от 20.10.2026; описание в `docs/culling/README.md` .

Что внутри

1. 0 · Словарик
2. 1 · Большая картина: что просили и что сделали
3. Шаг 1. Сцена из 10 000 объектов
4. Шаг 2. Реестр, который помнит
5. Шаг 3. Как реестр узнаёт об изменениях
6. Шаг 4. Синхронизация раз в кадр
7. Шаг 5. Пирамида видимости
8. Шаг 6. Коробка для бегущего человечка
9. Шаг 7. Из видимого набора в пакет
10. Шаг 8. Сортировка: два ключа
11. Шаг 9. Подача и счётчики
12. Шаг 10. Сцена не рисует
13. Шаг 11. Добавить, удалить, сдвинуть
14. Замеры: как мерили и что вышло
15. Проверка: картинка не изменилась
16. Тесты
17. Три сценария целиком
18. Слабые места и что сказать
19. Вопросы на защите: короткие ответы
20. Шпаргалка: что где лежит

0 · Словарик

Кадр	Одна картинка на экране. 60 кадров в секунду значит около 16,7 мс на кадр на всё: логику, анимацию, отрисовку.
Draw call (вызов отрисовки)	Команда видеокарте «нарисуй этот меш». На каждую процессор тратит время в драйвере.
Смена состояния	Переключение того, чем рисуем: программа шейдера, текстура. Каждое переключение стоит времени.
Bounds, AABB	Коробка вокруг объекта, стороны которой параллельны осям мира. Хранится двумя точками: min и max.
Мировая матрица	Таблица чисел, которая переводит объект из его собственных координат в координаты мира: сдвиг, поворот, масштаб, с учётом родителей.
Пирамида видимости (frustum)	Усечённая пирамида перед камерой: всё, что внутри, может попасть на экран. Задаётся шестью плоскостями.
Отсев (culling)	Отбрасывание объектов, которые точно не видны, до отрисовки.
Видимый набор (visible set)	Список объектов, прошедших отсев в этом кадре. У нас это список handle'ов.
Реестр	Подсистема, которая хранит записи об объектах между кадрами. У нас `Visibility::Service`.
Handle	Номерок записи в реестре. По нему обновляют и удаляют запись. Номера только растут и не повторяются.
Stateful / stateless	Stateful хранит свои данные между кадрами и меняет их по сообщениям. Stateless каждый кадр собирает всё заново из сцены.
Dirty (помечен)	Объект изменился и его запись в реестре надо обновить.
Пакет (submission)	Готовый отсортированный список того, что рисовать в этом кадре.
Front-to-back	Порядок «от ближних к дальним»: ближние закрывают дальних раньше, и видеокарта не тратит время на скрытые пиксели.
Скиннинг	Деформация меша костями скелета: так персонаж «шагает». Делается на видеокарте.
Медиана, p95, p99	Медиана: типичное значение (половина кадров быстрее). p95: 95 % кадров не медленнее этого. p99: то же для 99 %, то есть хвост и подтормаживания.

1 Большая картина: что просили и что сделали

ТЗ ЛР 3 предлагает пять вариантов. Наш вариант Д: culling + submission, отсев и пакетная подача.

ТЗ просит	Что сделано
Видимый набор как данные, посчитанный до отрисовки	`visible_` и `packet_` в `RenderSystem`
Отсев как подсистема с регистрацией, а не проход по сцене	`Visibility::Service`: register / update / unregister с handle
Изменения сообщаются подсистеме	запись в компонент помечает объект; синхронизация трогает только помеченных
Второй потребитель без правки	`query(матрица камеры)` не меняет реестр; тест спрашивает вторую камеру
Сцена не рисует	в игровом коде нет вызовов отрисовки, только компоненты
Пакет и минимум два ключа сортировки	шейдер, меш, текстура; затем глубина
Сцена ≥ 10 000 объектов, замеры до/после	9 900 кубов + 100 персонажей; 5 режимов × 3 прогона
Картинка не изменилась	36 попиксельных сравнений (на Windows все совпали)
RenderDoc-захват «после»	**нет в репозитории**, нужно снять на Windows

ПРОСТЫМИ СЛОВАМИ

Представьте склад и фотографа. Раньше фотограф каждый кадр обходил весь склад и снимал каждую коробку, даже те, что за спиной. Теперь у склада есть журнал, где записано, где лежит каждая коробка. Фотограф говорит, куда смотрит камера, журнал за долю секунды отвечает, какие коробки в кадре, и снимаются только они. Причём сначала все красные, потом все синие: так реже меняется плёнка.

Ш1 Сцена из 10 000 объектов

Тестовая сцена генерируется кодом. Это инструмент замера, а не игровой контент.

ПРОСТЫМИ СЛОВАМИ

Сетка 100 × 100 ячеек с шагом 4 метра, в каждой куб на одной из пяти высот. Кубы получают одну из четырёх текстур по кругу. Первые 100 объектов превращаются в анимированных персонажей Walking.fbx, они шагают на месте.

Номер ячейки считается как `(номер × 7919) mod 10000`. 7919 и 10000 взаимно просты, поэтому кубы разбрасываются по сетке, а порядок создания не совпадает с порядком по глубине. Иначе сортировка по глубине была бы «бесплатной» и замер ничего бы не показал.

ГДЕ В КОДЕ

- `src/editor/CullingScene.cpp` `loadCullingScene`: очистка мира, 100 групп иерархии, 10 000 объектов, 100 персонажей, камера.
- `src/editor/CullingScene.cpp:63` `addCullingObjects`: только `addComponent`, никакой отрисовки.
- `src/editor/panels/RendererInfoPanel.cpp` кнопка «Load 10,000-object scene», галочки, счётчики.
- `Запуск` `./build/mac/GameEngine --editor-culling --no-vsync` из корня репозитория.

ЕСЛИ СПРОСЯТ

Почему в редакторе Entities больше 10 000? Группы иерархии и камеры тоже сущности. В реестре отсева их нет, у них нет меша. Счётчик «Scene 10000 / GPU N» в окне Scene показывает именно реестр.

Зачем группы? 10 000 строк в окне Hierarchy неудобны. По 100 кубов в группе дерево остаётся рабочим; заодно это проверяет иерархию в отсеве.

Ш2 Реестр, который помнит

Главная деталь варианта Д. ТЗ прямо не засчитывает отсев, который каждый кадр заново обходит сцену.

ПРОСТЫМИ СЛОВАМИ

Охранник на входе ведёт журнал гостей. Пришёл гость: охранник пишет строчку и выдаёт номерок (это **регистрация** и **handle**). Гость пересел: охранник правит строчку по номерку (**update**). Ушёл: строчка вычеркивается (**unregister**). Если спросить «кто сидит у окна», охранник смотрит в журнал, а не обходит зал.

Журнал и есть stateful-подсистема: она хранит данные сама и между кадрами ничего не забывает.

```
struct Record { Entity entity; Bounds bounds; Mat4 model; bool alwaysVisible; };
class Service {
    Handle registerObject(Record);        // новый номерок: ++next_
    bool   update(Handle, Record);        // false, если номерок устарел
    bool   unregisterObject(Handle);
    std::vector<Handle> query(const Mat4& viewProjection, bool culling = true) const;
};
```

ГДЕ В КОДЕ

- `src/ecs/Visibility.h:56` `Visibility::Service`: реестр `records_` (handle → запись) и счётчик `next_`.
- `src/ecs/World.h` поле `World::visibility` и словарь `visibilityHandles` (сущность → handle).
- `src/ecs/Visibility.h:74` `query`: плоскости из матрицы, проверка каждой записи, список видимых.

ЕСЛИ СПРОСЯТ

Почему номерки только растут? Чтобы старый номерок не совпал с новым объектом. Даже `clear()` не сбрасывает счётчик. Тест «clear invalidates handles» это проверяет.

Есть ли быстрая структура вроде сетки или дерева? Нет, запрос проверяет все записи подряд. Для 10 000 это быстро. Структуру можно поставить внутрь `Service`, снаружи никто не заметит.

Почему query помечен const? Он только читает. Поэтому одна и та же подсистема обслуживает любую камеру: основную, вторую, миникарту, тест.

Ш3 Как реестр узнаёт об изменениях

Чтобы не обходить сцену, реестру нужно сообщать о каждом изменении. Здесь это сделано так, что забыть сообщить трудно.

ПРОСТЫМИ СЛОВАМИ

Представьте, что любая папка, которую взяли в руки «на правку», сама получает цветной стикер. Вечером архивариус проходит только по папкам со стикерами, а остальные не трогает.

В движке такой «правкой» считается доступ к компоненту на запись: `world.getComponent<Transform>(e)` без const. World сам кладёт сущность в набор изменённых. Читающему коду дают const-доступ или `forEachRead`, и он ничего не помечает.

```
template<typename T> void notifyComponentEdit(Entity e) {
    if constexpr (is_same_v<T,Transform> || is_same_v<T,MeshRenderer> ||
is_same_v<T,Hierarchy>)
        markSpatialDirty(e);   // spatialDirty_.insert(e)
}
```

ГДЕ В КОДЕ

- `src/ecs/World.h:31` `markSpatialDirty`, `takeSpatialChanges`.
- `src/ecs/World.h:134` `notifyComponentEdit`: зовётся из `getComponent`, `addComponent`, `removeComponent`, `forEach`.
- `src/ecs/World.h:125` `forEachRead`: обход только для чтения, без отметок.
- `src/ecs/AnimationSystem.cpp` помечает каждого анимированного персонажа: поза поменялась, коробка тоже.

ЕСЛИ СПРОСЯТ

Что если изменить Transform через ссылку, взятую раньше? Изменение не заметят. Поэтому правило в `World.h`: не хранить изменяемые ссылки между кадрами, а если очень нужно, звать `markSpatialDirty` вручную.

Почему помечаются только три типа? Только Transform, MeshRenderer и Hierarchy влияют на положение и коробку объекта. Правка Tag или скрипта отсев не касается.

Ш4 Синхронизация раз в кадр

Функция, которая переносит накопленные изменения из World в реестр.

ПРОСТЫМИ СЛОВАМИ

Архивариус берёт стопку папок со стикерами. Если среди них папка «отдел», он добавляет в стопку и все папки сотрудников отдела: переехал отдел, переехали все. Потом по каждой папке пишет новую строчку в журнал.

1. Забрать набор изменённых (`takeSpatialChanges`) и добавить объекты, у которых меш ещё грузится.
2. Для каждого с иерархией добавить в очередь его детей: у них поменялась мировая матрица.
3. Если у объекта нет меша, шейдера или он скрыт, снять его с учёта.
4. Посчитать мировую матрицу с учётом родителей и коробку по вершинам меша. Для анимированных взять коробку по позе (шаг 6).
5. Перевести коробку в мир и вызвать `registerObject` для нового или `update` для известного.

ГДЕ В КОДЕ

- `src/ecs/SceneVisibility.h:22` `SceneVisibility::synchronize` возвращает, сколько записей обновлено.
- `src/editor/EditorContext.cpp` `EditorContext::update` (строка 246) зовёт её каждый кадр, даже если окна сцены скрыты.
- `src/ecs/RenderSystem.cpp:36` ещё раз в `prepare` ; к этому моменту набор обычно пуст.

ЕСЛИ СПРОСЯТ

Сколько работы в неподвижной сцене? Ноль: набор изменённых пуст. Тест «10000 static no rescan» и скрипт замеров это проверяют: обновлений ровно 100, только анимированные.

А пока меш грузится? Рисуется куб-заглушка, а объект держится в списке «ждущих» и проверяется каждый кадр. Как только меш готов, коробка пересчитывается без всякой правки компонентов.

Ш5 Пирамида видимости

Как за несколько умножений понять, что коробка точно не видна.

ПРОСТЫМИ СЛОВАМИ

Встаньте в комнате с фонариком. Луч фонарика это пирамида. Чтобы понять, освещена ли коробка, не нужно проверять каждую её точку: достаточно для каждой стенки луча взять угол коробки, самый близкий к лучу с внутренней стороны. Если даже он снаружи хотя бы одной стенки, вся коробка в темноте.

У камеры шесть «стенок»: левая, правая, верхняя, нижняя, ближняя и дальняя. Их получают прямо из матрицы камеры: строка w плюс или минус строка x, y или z. Для каждой стенки берётся угол коробки, дальний по направлению внутрь. Это 6 проверок на объект.

```
bool intersects(Bounds b) const {
    for (const auto& p : planes) {
        float d = p.x*(p.x>=0 ? b.max.x : b.min.x) + p.y*(p.y>=0 ? b.max.y : b.min.y)
                + p.z*(p.z>=0 ? b.max.z : b.min.z) + p.w;
        if (d < -1e-4f) return false;   // даже «лучший» угол снаружи
    }
    return true;
}
```

ГДЕ В КОДЕ

- `src/ecs/Visibility.h:26` `Frustum` : плоскости из матрицы и проверка `intersects` .
- `src/ecs/Visibility.h:16` `transformed` : коробка в мир через 8 углов.

ЕСЛИ СПРОСЯТ

Может ли тест ошибиться? Только в безопасную сторону: у угла пирамиды коробка иногда считается видимой, хотя не видна. Тогда объект просто нарисуется. Потерять видимый объект тест не может.

Зачем порог −1e-4, а не 0? Объект, касающийся границы, считается видимым. Лучше нарисовать лишнее, чем увидеть мерцание на краю экрана.

Почему коробка, а не шар? Кубы и человечки вытянуты: шар был бы намного больше самого объекта, и отсев отбрасывал бы меньше.

Ш6 Коробка для бегущего человечка

Персонаж двигает руками и ногами, его коробка меняется каждый кадр. Скиннинг идёт на видеокарте, поэтому процессор не знает, где сейчас вершины.

ПРОСТЫМИ СЛОВАМИ

Представьте марионетку. Для каждой нитки (кости) заранее записано, какие части куклы она тянет, и вокруг этих частей нарисована своя маленькая коробка. В каждом кадре мы двигаем только маленькие коробки так же, как двигаются нитки, и обводим все вместе одной большой.

Почему вершина точно внутри: вершина после скиннинга это смесь положений «как если бы её тянула только кость 1», «только кость 2» и так далее, с весами от 0 до 1 в сумме 1. Смесь точек лежит между ними, а все они внутри своих коробок. Значит и смесь внутри общей коробки.

ГДЕ В КОДЕ

- `src/ecs/SkinBounds.h:12` `prepare`: один раз на подмеш строит коробку для каждой кости, кэш в `SubMesh::visibilityBounds`.
- `src/ecs/SkinBounds.h:34` `evaluate`: каждый кадр переводит коробки костей матрицами текущей позы и объединяет.
- `tests/AnimationTests.cpp` проверка на 64 фазах шага Walking.fbx: каждая вершина внутри коробки.

ЕСЛИ СПРОСЯТ

Что если веса кривые? Отрицательные веса или сумма не 1: кэш помечается небезопасным, и объект рисуется всегда (`alwaysVisible`). Неправильно отсечь хуже, чем нарисовать лишнее.

Почему не посчитать вершины на CPU? Это повторить скиннинг на процессоре для тысяч вершин 100 персонажей каждый кадр. Коробок костей десятки, их перевести на порядки дешевле.

Ш7 Из видимого набора в пакет

Рендер больше не рисует «по ходу обхода». Сначала он полностью готовит список, потом рисует.

ПРОСТЫМИ СЛОВАМИ

Повар сначала читает все заказы и раскладывает продукты по порядку на столе, и только потом включает плиту. А не бегает в кладовку за каждым ингредиентом отдельно.

1. **Камера.** Взять матрицы активной камеры.
2. **Синхронизация.** Применить накопленные изменения.
3. **Запрос.** `visible_ = world.visibility.query(projection × view)` : это visible set.
4. **Пакет.** Для каждого видимого записать: сущность, матрицу, программу шейдера, адрес меша, адрес текстуры, глубину центра коробки.
5. **Порядок.** Отсортировать по id (база), затем по ключам, если сортировка включена.

ГДЕ В КОДЕ

- `src/ecs/RenderSystem.cpp:33` `RenderSystem::prepare` .
- `src/ecs/RenderSystem.cpp:21` `render` : замеряет `prepareMs` и `submitMs` отдельно.
- `src/ecs/RenderSystem.h` структура `Submission` и счётчики `Stats` .

ЕСЛИ СПРОСЯТ

Почему сначала сортировка по id? Без сортировки порядок зависел бы от хэш-таблицы реестра и мог бы меняться между запусками. По id он стабилен, поэтому режимы честно сравнимы.

Что значит «отсев выключен»? `query` с `culling=false` возвращает все записи. Всё остальное работает так же, поэтому разница в замере это ровно цена или выигрыш отсева.

Ш8 Сортировка: два ключа

ТЗ требует минимум два осмысленных ключа и видимый эффект каждого.

ПРОСТЫМИ СЛОВАМИ

Художник красит забор. Если он берёт кисть с красной краской, красит одну доску, моет кисть, берёт синюю, потом снова красную, он больше моет кисти, чем красит. Разумнее сначала все красные доски, потом все синие: это **первый ключ**, группировка по ресурсу.

Внутри одного цвета он начинает с ближних досок. Ближние закрывают часть дальних, и на закрытое краска не тратится: это **второй ключ**, глубина от ближних к дальним. Видеокарта отбрасывает закрытые пиксели до шейдера (early-z).

Режим (Windows)	Смен текстур	Инверсий глубины
Только отсев	1765.5	16.5
+ группировка (без глубины)	16	989.5
+ глубина	16	0

ГДЕ В КОДЕ

- `src/ecs/RenderSystem.cpp:52` сортировка по кортежу `(shader, mesh, texture, depth, entity)`; флаг `depthSortingEnabled` убирает глубину.
- `src/ecs/RenderSystem.cpp:57` счётчик инверсий: соседи одной группы «дальний перед ближним».

ЕСЛИ СПРОСЯТ

Почему 16 смен текстур? В стенде замеров 8 текстур и 2 меша (куб и персонаж): 2 × 8 = 16 групп.

Почему не глубина первой? Тогда текстуры перемешались бы и смен снова стало бы много. Сначала дорогое состояние, потом порядок внутри.

Кадр стал быстрее от сортировки? Не доказано: полная сортировка не обогнала «только отсев» по медиане кадра. Текстуры в стенде 1×1, переключение дешёвое, а сама сортировка стоит времени. Эффект ключей виден в счётчиках, и это ТЗ допускает.

Ш9 Подача и счётчики

Подача идёт по готовому пакету. Состояние меняется, только если действительно изменилось.

ПРОСТЫМИ СЛОВАМИ

Кассир, который пробивает товары, не перенастраивает весы, если следующий товар тоже на весах. Он помнит, что сейчас включено.

```
if (boundProgram_ != program) { renderer_.useShaderProgram(program); boundProgram_ = program;
++stats_.shaderChanges; ... }
if (boundTexture_ != effectiveTexture) { renderer_.bindTexture2D(effectiveTexture, 0);
boundTexture_ = effectiveTexture; ++stats_.textureChanges; }
```

Счётчик в Renderer Info	Что значит
Registered / visible / submitted	в реестре / прошли отсев / реально нарисованы
CPU prepare / submit	время двух фаз, мс; в prepare включена синхронизация из update
Shader / texture switches	сколько раз реально переключили программу и текстуру
Dirty updates / depth inversions	сколько записей обновлено / «дальний перед ближним»
Оверлей Scene: «Scene N / GPU M»	живой счётчик «в сцене vs отправлено», которого требует ТЗ

ГДЕ В КОДЕ

- `src/ecs/RenderSystem.cpp:64` `submit` : обход пакета, матрица берётся из пакета.
- `src/ecs/RenderSystem.cpp:91` `drawEntity` : фильтр смены программы и отрисовка подмешей.
- `src/editor/panels/RendererInfoPanel.cpp` галочки и счётчики.
- `src/editor/panels/SceneViewPanel.cpp` строка «Scene / GPU».

ЕСЛИ СПРОСЯТ

Вызовов отрисовки стало меньше? Нет: один объект = один вызов (у персонажа по одному на подмеш). Меньше стало объектов в пакете и переключений состояния. Объединение одинаковых в один вызов (instancing) не сделано.

Ш10 Сцена не рисует

ТЗ проверяет это «тычком в файл»: выбирают игровой файл и смотрят, что в нём нет отрисовки.

> **ПРОСТЫМИ СЛОВАМИ**
>
> Режиссёр (игровой код) говорит актёрам, где стоять. Оператор (рендер) снимает. Режиссёр камеру в руки не берёт.

Файл	Есть отрисовка?
`src/editor/CullingScene.cpp`	нет, только компоненты
`assets/scripts/*.lua`	нет, в API Lua нет отрисовки
`src/ecs/AnimationSystem.cpp`, `Visibility.h`, `SceneVisibility.h`	нет
`src/ecs/RenderSystem.cpp`	да, через `IRenderAdapter`
`src/render/OpenGLRenderAdapter.cpp`	да, это слой OpenGL
`src/bench/CullingBenchmark.cpp`	`glReadPixels` для сравнения кадров, вне замеров

ГДЕ В КОДЕ

- Проверка поиском `grep -rnE "drawIndexed|useShaderProgram|bindTexture2D" src | grep -v src/render/` находит только `RenderSystem.cpp`.

Ш11 Добавить, удалить, сдвинуть

Живой критерий: объекты меняются на ходу через регистрацию, без пересканирования сцены.

Действие	Что происходит
Add 100	`createEntity` + `addComponent`: сущности помечены, следующая синхронизация зарегистрирует 100 записей
Delete 100	`destroyEntity` сразу вызывает `unregisterObject` и чистит служебные наборы
Move 100 objects	`getComponent<Transform>` каждый кадр: 100 обновлений, остальные 9 900 не трогаются
Сдвинуть группу	родитель помечен, синхронизация добавляет всех детей
Load 10,000-object scene	полная перезагрузка: `world.clear()` очищает реестр, объекты создаются заново

ГДЕ В КОДЕ

- `src/ecs/World.cpp:46` `destroyEntity`: снятие с учёта сразу.
- `src/editor/CullingScene.cpp:95` `removeCullingObjects`.
- `src/states/EditorState.cpp` проверка `--editor-culling-check`: те же команды между настоящими кадрами, отчёт в `docs/culling/animated/editor-check.csv` (10 проверок, все PASS).

ЕСЛИ СПРОСЯТ

Load scene это же пересоздание структуры? Да, но это осознанная полная перезагрузка по кнопке. Требование «без пересоздания с нуля» относится к добавлению, удалению и движению, а они идут через регистрацию.

3 Замеры: как мерили и что вышло

ТЗ: Release, прогрев, N ≥ 3, медиана и p95/p99, «до» с принудительно выключенным отсевом.

- Отдельный процесс на каждый прогон, VSync выключен, скрытое окно 1280×720.
- Прогрев 120 кадров, затем 360 измеряемых; 3 прогона на каждый из 5 режимов; порядок режимов сдвигается.
- Перед показом кадра `glFinish`: время кадра включает ожидание видеокарты и не прячет очередь.
- Камера летит по фиксированному маршруту, шаг анимации фиксирован, все режимы видят одинаковые кадры.
- Медиана, p95, p99 по 1 080 кадрам режима. CPU рендера = prepare + submit, без расчёта поз; это не время GPU.

Windows · NVIDIA RTX 3050 Laptop · данные однокурсника

Режим	Кадр, медиана	Кадр, p95	CPU рендера, медиана	Отправлено	Смен текстур	Инверсий
Отсев выкл., сортировка выкл.	21,43	25,55	18,53	10000	10000	0
Только сортировка	20,73	25,88	16,97	10000	16	0
Только отсев	6,69	8,58	3,89	1784	1765.5	16.5
Отсев + группировка без глубины	7,82	10,74	3,90	1784	16	989.5
Отсев + полная сортировка	7,01	9,64	3,80	1784	16	0

Кадр 21,43 → 7,01 мс (в 3,05 раза), CPU рендера 18,53 → 3,80 мс (в 4,87 раза).

Mac · Apple M1 Pro · прогон на этом ноутбуке 20.10.2026

Режим	Кадр, медиана	Кадр, p95	CPU рендера, медиана	Отправлено	Смен текстур	Инверсий
Отсев выкл., сортировка выкл.	46,25	50,75	43,75	10000	10000	0
Только сортировка	45,84	50,54	42,86	10000	16	0

Режим	Кадр, медиана	Кадр, p95	CPU рендера, медиана	Отправлено	Смен текстур	Инверсий
Только отсев	16,30	23,20	13,35	1784	1765.5	16.5
Отсев + группировка без глубины	15,87	23,86	12,71	1784	16	989.5
Отсев + полная сортировка	16,01	23,91	12,82	1784	16	0

Кадр 46,25 → 16,01 мс (в 2,89 раза), CPU рендера 43,75 → 12,82 мс (в 3,41 раза). Окно на Retina даёт буфер 2560×1440, поэтому абсолютные числа с Windows не сравнивать.

ГДЕ В КОДЕ

- `tools/run_culling_bench.py` матрица режимов и `summary.json`.
- `src/bench/CullingBenchmark.cpp` стенд: сцена, маршрут камеры, CSV.
- `docs/culling/animated/` CSV, `summary.json`, проверки.

ЕСЛИ СПРОСЯТ

Почему медиана, а не среднее? Редкие длинные кадры сдвигают среднее. Медиана показывает типичный кадр, p95 и p99 показывают подтормаживания.

Почему кадр ускорился меньше, чем CPU рендера? В кадр входят расчёт поз, ожидание видеокарты и прочее, что отсев не трогает.

П Проверка: картинка не изменилась

Отсев не должен терять видимые объекты. Это проверяют попиксельно.

ПРОСТЫМИ СЛОВАМИ

Сделать две фотографии одной и той же сцены: одну «как раньше» (всё выключено), другую с отсевом. Наложить и сравнить каждый пиксель.

Стенд снимает 12 положений камеры. В каждом: эталон «отсев и сортировка выключены» и три других режима, всего 36 сравнений. Во второй половине добавляются 100 временных объектов с поворотом и отрицательным масштабом. Отличие больше 2 на канал считается ошибкой, и процесс завершается с кодом 1.

Windows: все 36 сравнений без единого отличающегося пикселя.

Mac (M1 Pro): 34 из 36. В одной позиции камеры при включённой сортировке отличается 1 пиксель. Отсев без сортировки совпадает везде. Причина: когда две поверхности в пикселе дают одинаковую глубину, побеждает та, что нарисована первой, а сортировка меняет порядок. Это «ничья» по глубине, отсев к ней не причастен.

ГДЕ В КОДЕ

- `src/bench/CullingBenchmark.cpp` `verifyImages` : чтение кадра `glReadPixels` и сравнение.
- `docs/culling/animated/visual-check.csv` результаты Windows.

ЕСЛИ СПРОСЯТ

Что сказать, если упадёт на Mac? Показать, что расходится один пиксель и только при сортировке; объяснить «ничью» по глубине. Исправление на будущее: строгий порядок при равной глубине или допуск по числу пикселей.

Тесты

12 наборов ctest; новый набор `VisibilityTests`.

Что проверяется	Где
внутри / снаружи / касание плоскости / камера внутри объекта	VisibilityTests
устаревший handle, `clear` делает handle недействительными	VisibilityTests
движение, отрицательный масштаб, скрытие и повторная регистрация, удаление компонента	VisibilityTests
иерархия: сдвиг и удаление родителя	VisibilityTests
10 000 статических объектов без повторного обхода; вторая камера	VisibilityTests
ресурс догрузился, поза анимации, нулевые и отрицательные веса	VisibilityTests
каждая вершина Walking.fbx внутри коробки на 64 фазах	WalkingAnimationTests

ГДЕ В КОДЕ

- `tests/VisibilityTests.cpp` все проверки подсистемы отсева.
- `Запуск ctest --test-dir build/mac --output-on-failure` : на этом Mac 12 из 12 проходят.

C Три сценария целиком

Если попросят рассказать «что происходит, когда…».

1. Обычный кадр, камера летит

1. `AnimationSystem` считает позы и помечает 100 персонажей.
2. `EditorContext::update` зовёт `synchronize`: 100 обновлений, кубы не трогаются.
3. Окно Scene: `renderSystem.render` → `prepare`: `query` по матрице камеры, около 1 800 видимых.
4. Пакет сортируется по ресурсам и глубине; `submit` рисует, переключая текстуру около 16 раз.
5. Статистика попадает в Renderer Info и в строку «Scene / GPU».

2. Нажали Add 100

1. `addCullingObjects(100)` создаёт сущности и добавляет компоненты; World помечает их.
2. Следующий `synchronize` регистрирует 100 записей: Registered 10 000 → 10 100.
3. Остальные 10 000 записей не пересчитываются.

3. Сняли галочку Frustum culling

1. `renderSystem.cullingEnabled = false`.
2. `query` возвращает все записи: visible = submitted = 10 000.
3. CPU prepare и submit растут, кадр тяжелеет: разница видна в счётчиках сразу.

! Слабые места и что сказать

Лучше назвать самим.

- **RenderDoc-захвата нет.** ТЗ требует захват «после» у GL-команд. На macOS RenderDoc с OpenGL не работает, снять на Windows.
- **Допфичи варианта Д не сделаны:** нет многопоточной сборки видимого набора, instancing, отсева по расстоянию и occlusion. Можно попробовать заявить коробки анимации, проверку картинки с кодом возврата и зоны Tracy, но в таблице допфич их дословно нет.
- **Индекс линейный**, O(N). Для 10 000 быстро; сетка или BVH ставятся внутрь `Service`.
- **Один объект = один вызов отрисовки.** Сокращаются переключения, а не вызовы.
- **Ускорение кадра от сортировки не доказано**, эффект виден в счётчиках.
- **На Mac проверка картинки падает** на одном пикселе из-за равной глубины.

? Вопросы на защите: короткие ответы

Что такое visible set у вас?
Список handle'ов `visible_`, который `prepare` получает у реестра до отрисовки.

Чем ваш отсев stateful?
Реестр хранит записи между кадрами, меняет их только по сообщённым изменениям, а запрос читает только свои записи, не World.

Как подсистема узнаёт, что объект сдвинули?
Доступ к Transform на запись сам помечает сущность; раз в кадр синхронизация обрабатывает помеченных.

Что будет при удалении объекта?
`destroyEntity` сразу вызывает `unregisterObject`.

Где второй потребитель?
`query(матрица)` const: тест спрашивает вторую камеру, стенд замеров пользуется тем же реестром.

Какие ключи сортировки и зачем?
Программа, меш, текстура (меньше переключений), затем глубина (ближние первыми, early-z).

Где видно эффект каждого ключа?
Смен текстур 1 765 → 16 от группировки; инверсий 989 → 0 от глубины.

Сколько объектов и сколько уходит на GPU?
10 000 в сцене, на стенде около 1 784 видимых в медиане.

Какая цена подсистемы?
В статике 0 обновлений и один проход запроса по 10 000 записей; итоговый CPU рендера 3,8 мс на Windows.

Как проверили, что картинка не изменилась?
36 попиксельных сравнений с режимом «всё выключено»; на Windows все совпали.

Почему AABB, а не сфера?
Объекты вытянуты, AABB плотнее и считается дёшево.

Как работает проверка коробки?
Для каждой из 6 плоскостей берётся угол коробки, дальний по нормали; если он снаружи, коробка невидима.

Откуда плоскости?
Из матрицы projection × view: строка w плюс или минус строки x, y, z.

Как отсекаются анимированные персонажи?
Коробки костей переводятся матрицами позы и объединяются; это гарантированно содержит все вершины.

Что если ресурс ещё грузится?
Рисуется заглушка, объект в списке ожидания, коробка обновится после загрузки.

Почему стабильный порядок по id без сортировки?
Чтобы порядок не зависел от хэш-таблицы и режимы были сравнимы.

Где в игровом коде вызовы отрисовки?
Нигде; рисует только RenderSystem через адаптер.

Почему не BVH?
Не требовалось; для 10 000 линейный проход быстрый, а интерфейс позволяет поставить BVH внутрь.

Что такое early-z?
Видеокарта отбрасывает фрагмент до шейдера, если он дальше уже записанной глубины.

Почему для прозрачных был бы обратный порядок?
Смешивание требует рисовать дальние первыми, иначе цвет получится неверный.

⌘ Шпаргалка: что где лежит

Файл	Что там
`src/ecs/Visibility.h`	Service, Record, Frustum, Bounds
`src/ecs/SceneVisibility.h`	synchronize, worldMatrix
`src/ecs/SkinBounds.h`	коробки анимации
`src/ecs/World.h / World.cpp`	отметки изменений, forEachRead, снятие с учёта при удалении
`src/ecs/RenderSystem.h / .cpp`	prepare, submit, счётчики
`src/editor/CullingScene.cpp`	сцена 10 000, Add / Delete
`src/editor/panels/RendererInfoPanel.cpp`	галочки и счётчики
`src/bench/CullingBenchmark.cpp`	стенд замеров, сравнение кадров
`tools/run_culling_bench.py`	матрица режимов, медиана и перцентили
`tests/VisibilityTests.cpp`	тесты подсистемы
`docs/culling/README.md`	описание и итоговая таблица однокурсника

Запуск редактора: `./build/mac/GameEngine --editor-culling --no-vsync` · окно стенда: `--culling-demo --culling-animated` · тесты: `ctest --test-dir build/mac --output-on-failure`